

Bots & Bytes — Setup Guide

Read this before the workshop. Or read it during the first 30 minutes — that block is reserved for setup and we'll go at the pace of the room. Either way, this is the document that gets you to the point where you can build with the rest of us.

If a step looks weird, that's normal. Computers are fussy. Flag your neighbor or the facilitator.

0. Welcome — what to expect

Audience: This guide assumes nothing. If you've never opened a terminal before, you're in the right place. Every step has a Mac path and a Windows path.

Time to complete: 20–40 minutes the first time. Less if you already have some of these tools.

You will install:

1. **Git** — version-control tool, also gives Windows users a working terminal
2. **Python 3.11 or newer** — programming language the bot is built in
3. **Visual Studio Code (VS Code)** — code editor with a built-in terminal
4. **Claude Desktop** — the AI tool you'll use to drive the bot
5. **A Mangrove API key** — credentials for the trading service (free signup)
6. **The workshop repo** — clone it from GitHub, run one setup script

You will need: - A laptop (Mac or Windows). Linux works too but isn't covered here. - An internet connection. - About 2 GB of free disk space. - A Claude Pro, Max, or Team subscription (or API credits) — Claude Code requires this. - (Optional, for Part C of the workshop) ~\$5 USDC on Base mainnet for live trading. You can do the workshop without this.

What is a “terminal” anyway?

A **terminal** is a window where you type text commands instead of clicking buttons. Sometimes called the “command line,” “shell,” or “console.” Looks intimidating; it isn't.

Think of it like text-messaging your computer. You type, hit Enter, the computer types back.

You'll need one because some installation steps and the workshop's setup script run as terminal commands. We'll open it together in Step 1.

On Mac: the terminal is built in. It's an app called **Terminal** in /Applications/Utilities/, or press Cmd + Space and type “terminal.”

On Windows: the built-in terminals (PowerShell, CMD) don't play well with the Unix-style commands the workshop uses. You'll get **Git Bash** when you install Git in Step 2 — that's the terminal you'll use.

Copy-pasting into a terminal: highlight the command, then Cmd+C (Mac) or Ctrl+C (Windows) to copy. Click into the terminal window. To paste: Cmd+V on Mac, Right-click or Shift+Insert on Windows Git Bash. (The usual Ctrl+V doesn't work in most terminals.)

Step 1: Open a terminal

On Mac: 1. Press Cmd + Space to open Spotlight Search. 2. Type terminal and hit Enter. 3. A black or white window opens. Leave it open — you'll come back to it.

On Windows: You don't have a usable terminal yet. Go to Step 2 (Git) — installing Git gives you Git Bash, which becomes your terminal. Come back here after Step 2.

Step 2: Install Git

What it is: Git is the standard tool developers use to track changes in code and copy code from places like GitHub. We'll use it to download the workshop repo. On Windows, the Git installer also includes **Git Bash**, which is the terminal you'll need for everything else.

Why we need it: To clone the workshop repo. Plus, Claude Code on Windows requires Git to be installed.

On Mac

Mac usually ships with Git. Check if you already have it:

1. In Terminal, type `git --version` and press Enter.
2. If you see something like `git version 2.x.x`, you're done with this step. Skip to Step 3.
3. If you see a prompt asking to install Xcode Command Line Tools, click **Install** and wait. Re-run `git --version` when it finishes.
4. If the command isn't found, install Git from the official installer:
 - Go to <https://git-scm.com/download/mac>
 - Follow the "binary installer" link, download, and double-click to install.
 - Re-run `git --version` to verify.

On Windows

1. Go to <https://git-scm.com/download/win>
2. The download starts automatically. When it finishes, open the .exe file.
3. Click through the installer. **Defaults are fine for everything** — including the editor choice, the path option, and the line-ending option.
4. When it finishes, open **Git Bash** from your Start menu. (Search for "Git Bash" if you don't see it.)
5. In Git Bash, type `git --version` and press Enter. You should see `git version 2.x.x`.

Use Git Bash (not PowerShell or CMD) for every terminal step in this guide.

If it didn't work: - "git: command not found" → installer didn't add Git to PATH. Reinstall and accept the default PATH option. - Windows: you opened CMD or PowerShell instead of Git Bash. Open Git Bash specifically.

Official install reference: git-scm.com/install

Step 3: Install Python 3.11 or newer

What it is: Python is the programming language the trading bot is written in. The setup script we'll run later needs Python.

Why we need it: The bot's server (./scripts/setup.sh) builds a Python environment to run.

Note: We need version 3.11 or newer. Python 3.11 itself is "security-fixes only" mode now — the latest stable is 3.13. Either works. The latest version on python.org is fine.

On Mac

1. Open <https://www.python.org/downloads/macos/>
2. Download the latest "macOS 64-bit universal2 installer" (any 3.11+ version).
3. Open the downloaded .pkg file and click through the installer.
4. **Important:** at the end, the installer may show a Finder window with a script called `Install Certificates.command` — double-click it to install certificates Python needs to talk to the internet.
5. Open **Terminal** (Cmd + Space, type "terminal"). Verify:

```
python3 --version
```

You should see something like Python 3.13.1 (any 3.11+).

On Windows

1. Open <https://www.python.org/downloads/windows/>
2. Download the "Windows installer (64-bit)" for the latest 3.x version.
3. Open the .exe. **CRITICAL:** before clicking Install, **check the box that says "Add python.exe to PATH"** at the bottom of the installer window. Without this, Python won't work in your terminal.
4. Click **Install Now** and wait.
5. Open **Git Bash**. Verify:

```
python --version
```

You should see something like Python 3.13.1.

If python doesn't work, try python3 or py. Note which one works — you'll use it later. The setup script auto-detects.

If it didn't work: - "python: command not found" on Windows → you forgot to check "Add to PATH." Uninstall and reinstall with the box checked. - "command not found" on Mac → log out and log back in to refresh PATH, then retry.

Official install reference: [python.org downloads](https://python.org/downloads) · [Using Python on macOS](#)

Step 4: Install Visual Studio Code (VS Code)

What it is: A free code editor from Microsoft. Open-source. Works on Mac and Windows. We'll use it because it has a great built-in terminal — you can type commands and edit code in the same window.

Why we need it: It's our editor for the workshop. Its integrated terminal is also more comfortable than a bare Terminal/Git Bash window.

On Mac

1. Go to <https://code.visualstudio.com/download>
2. Click the macOS download button (universal — works on Apple Silicon and Intel).
3. Open the downloaded .zip. A Visual Studio Code.app appears in your Downloads folder.
4. Drag Visual Studio Code.app into your Applications folder.
5. Open it. Click **Trust** if macOS warns about an app from the internet.
6. Make the code command work from terminal:
 - In VS Code, press `Cmd + Shift + P` to open the Command Palette.
 - Type `Shell Command: Install 'code' command in PATH` and press Enter.
 - You should see a confirmation message.

7. Verify in Terminal:

```
code --version
```

You should see something like `1.95.0`.

On Windows

1. Go to <https://code.visualstudio.com/download>
2. Click the Windows download.
3. Open the .exe and click through the installer. **Defaults are fine.** The installer adds VS Code to your PATH automatically.

4. Open VS Code from the Start menu. Click **Trust** if prompted.
5. Restart Git Bash (close and reopen it) so it picks up the new PATH.
6. Verify in Git Bash:

```
code --version
```

You should see something like 1.95.0.

Open VS Code's terminal: Inside VS Code, press `Ctrl + `` (backtick — usually under the Esc key) on Windows, or `Cmd + `` on Mac. A terminal panel opens at the bottom of the window. **You will use this terminal for the rest of setup.**

On Windows: VS Code may default to PowerShell. Switch to Git Bash in the terminal panel: click the dropdown arrow next to the + icon in the terminal toolbar, choose **Git Bash**.

Official install reference: [VS Code on Mac](#) · [VS Code on Windows](#)

Step 5: Install Claude Desktop (includes the Claude Code CLI)

What it is: The desktop app for Claude — Anthropic's AI assistant. **It bundles the Claude Code CLI**, which is what we'll actually drive the trading bot with. (The CLI is the terminal-based agent. The Desktop app is its visual home.)

Why we need it: This is the AI tool you'll be talking to.

Prerequisite: A Claude Pro, Max, Team, or Enterprise subscription. Or API credits. If you don't have either, sign up at claude.com/pricing before continuing.

On Mac and Windows

1. Go to <https://claude.com/download>
2. Download the version for your OS. The Mac version is “universal” (works on Apple Silicon and Intel).
3. Open the installer:
 - **Mac:** open the .dmg, drag Claude into Applications, eject the disk image, open Claude from Applications.
 - **Windows:** run the .exe, click through.
4. Sign in with your Claude account.
5. Verify Claude Code CLI is installed. Open VS Code's integrated terminal (Step 4) and run:

```
claude --version
```

You should see something like `c\laude X.Y.Z`.

On Windows: you may need to close and reopen Git Bash / VS Code for the `c\laude` command to appear on PATH.

If `c\laude --version` doesn't work: - Open Claude Desktop once and let it finish first-run setup. Then reopen your terminal. - On Windows: confirm Git is installed (Step 2). Claude Code on Windows requires Git.

Official references: [Download Claude](#) · [Get started with the desktop app](#)

Step 6: Get a Mangrove API key

What it is: A credential that lets the bot talk to Mangrove's trading API.

Why we need it: Without it, the bot can't fetch market data or run strategies.

How:

1. Go to <https://mangrovedeveloper.ai>
2. Sign up for an account (email + password).
3. After logging in, find the **API Keys** section in your dashboard.
4. Generate a new key. It looks like a long string of random characters.
5. **Copy it somewhere safe** — you'll paste it once during Step 8. Treat it like a password; don't share it, don't commit it to GitHub.

If the workshop facilitator is handing out keys at the door, you can skip this step.

Step 7: Clone the workshop repo

This is the first step where you actually use the terminal commands you've been preparing for.

1. **Open VS Code.**
2. Open VS Code's integrated terminal (`Ctrl + `` on Windows, `Cmd + `` on Mac).
3. Decide where on your computer you want the workshop folder to live. Most people use Desktop. To go there:

```
cd ~/Desktop
```

(`cd` means "change directory" — it moves your terminal's "current location.")

4. Clone the repo:

```
git clone https://github.com/MangroveTechnologies/mangrove-agent.git mangrove-agent
```

This downloads the entire workshop project into a folder called `mangrove-agent`. Takes 10–30 seconds.

5. Move into that folder:

```
cd mangrove-agent
```

6. Open the folder in VS Code so you can see the files:

```
code .
```

The dot at the end means “the current folder.”

You should now see the project’s file tree on the left side of VS Code.

Step 8: Run the setup script

This is the one command that wires everything together. It installs the bot’s Python dependencies, starts the local server, and registers it with Claude Code.

1. In VS Code’s terminal (still inside the mangrove-agent folder), run:

```
./scripts/setup.sh
```

2. The script will ask for your Mangrove API key (from Step 6). Paste it and press Enter.

3. Wait ~60 seconds. You’ll see lines about creating a virtual environment, installing dependencies, starting the server, and registering with Claude.

4. **When you see this output, you’re done:**

```
==> Done. Setup complete.  
Agent: http://localhost:9080  
PID: 12345
```

If the script errors out, jump to the **Troubleshooting** section below.

Step 9: Verify you’re ready

Last sanity check.

1. In VS Code’s terminal (in the mangrove-agent folder), run:

```
c laude
```

2. Wait. Within ~30 seconds, the bot should run a **5-tool greeter** — five tool calls, each shown inline, followed by an invitation to build a strategy. It looks something like:

```
Hey — I’m your local Mangrove-powered trading bot...
```

```
[calls status] Bot is alive. Version 0.1.0...
```

[calls list_tools] Capability surface: wallets, market data, strategies...

Want me to build you a strategy? Tell me the asset and the vibe...

3. If you see that, **you're set**. Quit Claude with `Ctrl + C` (or just close VS Code) — we'll start fresh during the workshop.

If the greeter doesn't fire, see **Troubleshooting**.

Troubleshooting

git --version says "command not found"

Step 2 didn't complete. Reinstall Git from git-scm.com/install. On Windows, accept the default PATH option in the installer.

python --version (or python3) says "command not found"

Step 3 didn't complete or PATH wasn't updated. - **Windows**: uninstall Python from Settings → Apps, then reinstall — and **check "Add python.exe to PATH"** before clicking Install Now. - **Mac**: open a fresh Terminal window (PATH updates only on new terminals).

Setup script errors with "permission denied: ./scripts/setup.sh"

The script doesn't have execute permission. Run:

```
chmod +x ./scripts/setup.sh
./scripts/setup.sh
```

Setup script errors with "python3 not on PATH" or "Python 3.11 required"

You either don't have Python installed or have an older version. Re-do Step 3 with a Python 3.11+ installer.

Setup script errors mentioning port 9080

Something else on your computer is using port 9080. Find and kill it: - **Mac/Linux**:
`lsof -ti:9080 | xargs kill -9` - **Windows Git Bash**:
`netstat -ano | grep 9080` to find the process ID, then `taskkill /PID <id> /F`

Then rerun `./scripts/setup.sh`.

Setup script errors with “MANGROVE_API_KEY is still the placeholder”

The prompt for your API key didn't take. Open the file `server/src/config/local-config.json` in VS Code, find the `MANGROVE_API_KEY` field, and paste your key in directly. Save the file. Rerun `./scripts/setup.sh --yes --no-mcp --no-verify`.

claude opens but the greeter never fires

The bot's MCP server isn't registered with Claude Code. Run:

```
claude mcp list
```

If you see `mangrove-agent: ... x Failed to connect`, fix it:

```
claude mcp remove mangrove-agent  
./scripts/setup.sh --yes --no-verify
```

Then close and reopen Claude.

Anything else goes weird — the nuclear reset

Wipes everything and starts over (you lose any local work):

```
kill $(cat agent-data/bare.pid) 2>/dev/null  
rm -rf agent-data/  
claude mcp remove mangrove-agent 2>/dev/null  
./scripts/setup.sh
```

What success looks like

By the time you arrive at the workshop, you should be able to:

☐

Open a terminal (Terminal on Mac, Git Bash or VS Code's integrated terminal on Windows)

☐

Run `git --version` and see a version number

☐

Run `python --version` (or `python3 --version`) and see 3.11 or newer

☐

Run `code --version` and see a version number

☐

Run `claude --version` and see a version number

☐

Run `claude` inside the `mangrove-agent` folder and see the 5-tool greeter

If all six are green, you're ready to build.

Safety reminders before live trading (Part C only)

- **Never paste a private key into the Claude chat.** The bot has a hook that should refuse it; don't rely on the safety net.
 - **Paper before live, always.** Run paper mode for at least a full timeframe cycle before going live.
 - **Start with \$1–5 USDC for your first live swap.** Scale up only after you've watched a few clean fills.
 - **Commit your code before risky operations.** If something goes wrong, you can `git reset` and recover.
-

Reference materials

Everything below is open and free. Bookmark what's useful.

Mangrove on GitHub — open source

The Mangrove organization on GitHub: <https://github.com/MangroveTechnologies>

- [mangrove-agent](#) — the workshop repo. The thing you cloned in Step 7. Includes the tutorial chapters, docs, and the bot's source code.
- [MangroveKnowledgeBase](#) — the open-source signal library. 223 trading signals + 40+ technical indicators. Read the code, fork it, contribute.
- [MangroveMarkets](#) — the open-source markets infrastructure (the layer behind the mangrovemarkets Python SDK).
- [mangrove-trader-plugin](#) — Claude Code plugin / TypeScript SDK for connecting to Mangrove via MCP.
- [x402-app-template](#) — template for building x402-payment-gated apps.
- [x402-plugin-template](#) — template for building x402-aware Claude Code plugins.

Python packages on PyPI

The packages your workshop bot installs into its virtualenv. Each one is open source — clicking the link below shows the version, the readme, and a link back to the source repo.

- [mangrove-kb](#) — the signal library. `pip install mangrove-kb`
- [mangroveai](#) — the AI/intelligence SDK. `pip install mangroveai`
- [mangrovemarkets](#) — the markets/execution SDK. `pip install mangrovemarkets`

Inside the workshop repo (mangrove-agent)

Once you've cloned the repo (Step 7), these are the files worth knowing about. All in your `mangrove-agent/` folder.

Top-level orientation: - [README.md](#) — repo intro, quick start, layout overview - [CLAUDE.md](#) — what Claude Code reads on session start; defines conventions, agent identity, the bot's mental model - [CONTRIBUTING.md](#) — how to contribute back

Tutorial walkthrough — `tutorials/trading-app/`: - [00-index.md](#) — overview of the 8-chapter arc - [01-claude-code-and-ai-safety.md](#) — Claude Code mental model, three safety nets - [02-overview.md](#) — what the bot is, where state lives, the trust boundary - [03-setup.md](#) — long-form version of this setup guide - [04-your-first-strategy.md](#) — author + backtest a strategy - [05-paper-mode.md](#) — promote to paper, force a tick, read the audit trail - [06-wallet-setup.md](#) — create or import a wallet, encryption model, backup gate - [07-going-live.md](#) — promote to live, watch a real swap, verify on basescan - [08-monitor-troubleshoot-extend.md](#) — pause/archive/roll, debug, extend with custom signals + MCP tools + hooks

Reference docs — `docs/`: - [api-reference.md](#) — full REST API surface - [architecture.md](#) — system diagrams, module layout, design decisions - [specification.md](#) — API contracts and data models - [strategy-lifecycle.md](#) — deep dive on scheduler, jobstore, cron ticks, signal evaluation - [configuration.md](#) — config files and environment variables - [workshop-prereqs.md](#) — original short-form prereqs (this guide is the expanded version)

Where else to look

- **Claude Code docs:** code.claude.com/docs
 - **Mangrove developer site:** mangrovedeveloper.ai
 - **Git docs:** git-scm.com/doc
 - **Python docs:** docs.python.org/3
 - **VS Code docs:** code.visualstudio.com/docs
-

Where to get help

- **During the workshop:** flag your neighbor first (buddy system), facilitator second.
 - **Before the workshop:** email tim.darrah@mangrove.ai
 - **GitHub issues:** if you find a bug while doing the tutorial, open an issue on the relevant repo. The Mangrove team reads every one.
 - **For specific topics:** the tutorial chapters above are the long-form companion to each section of the workshop.
-

One last thing

If you get most of the way through this guide and something is broken, **don't try to fix it for hours alone**. Show up to the workshop with whatever you have — we'll spend the first 30 minutes pair-debugging anything that's stuck. The buddy system works.

See you Friday.

— Tim